

FlyWise — APIx Master Build Plan

FlyWise is the product/dashboard name; APIx remains the technical name of the index itself. **Windows dev machine • Solo build • Cloud-hosted • 5-6 day sprint build** (compressed from the PS's own 20-day reference timeline — see §0a)

This is the single source of truth for the build. Every decision made across our planning conversations is consolidated here. No code yet — this is the map; you'll ask for implementation prompts step by step.

0. What's already locked (recap)

Decision	Value
Route basket	12 core routes + 2 seasonal analytics routes (see §5)
Index formula	Jevons geometric mean at route level, Young-type weighted aggregation for DAPIx/IAPix
Currencies	INR (primary — all domestic routes and the index itself), AED (Dubai routes), SGD (Singapore routes), USD (display-layer conversion only, for international users — never enters the index calc)
Constant-quality method	Exact-match via Fare Specification ID (no hedonic model)
Primary data source	Bright Data — Google Flights Scraper API (5,000 free credits/month)
Secondary source	Scrappa — Google Flights JSON (500 free credits/month)
Tertiary source	1-2 direct OTA scrapes (Ixigo/EaseMyTrip, plain Playwright, no anti-bot tooling)
Collectors + ingestion	Hosted (Render Cron Job) — not run from your laptop, keeps collecting even when your laptop is off
Ingestion/normalization/dedup	Python (merged into the same hosted pipeline — no separate ETL tool)
Everything downstream	Python (quality/FX rules, index engine)
Backend	FastAPI
Policy/institutional API	Same FastAPI service, API-key-authenticated endpoints + auto-generated OpenAPI docs + CSV export — this is the PS's explicit "API that NSO and RBI can consume" requirement (see §8a)
Frontend	React/Next.js + TailwindCSS + Framer Motion + a 3D globe library (see §1a)
Hosting	Cloud (this doc specifies exact services below)

0a. The real timeline — 5-6 day sprint (not 20)

Everything below in §1-§10 is unchanged in *content* — same routes, same edge cases, same stack. What changes is sequencing: things that were sequential in a 20-day plan now run in

parallel where possible, and the base period compresses to Day 1 (see the note below the table).

Day	What gets done	Why this order
Day 1	All accounts + installs (§2, §3) done first thing. DB schema (§5) deployed to Neon immediately after. Kick off 2-3 manual collection test runs against Bright Data/Scrappa the same day — this becomes your base period.	Everything downstream needs the DB and at least one real API response to exist. No parallel work possible yet — this is the one truly sequential day.
Day 2	Build all three collectors (Bright Data, Scrappa, OTA Playwright) plus the ingestion/normalization/dedup scripts (§8) as one Python pipeline. Deploy it as a Render Cron Job running every 4 hours for both domestic and international routes.	Collectors and ingestion logic are both plain Python now (no separate ETL tool), so they're built and deployed together in one pass instead of two.
Day 3	Build the Python processing layer: quality/FX engine, the 4-tier imputation hierarchy, index engine (Jevons + Young, DAPIx/IAPix split). Wire in the edge cases (diverted-flight logging, matched-replacement substitution, glitch-fare flagging, codeshare dedup) directly into this layer as you build it — not as a separate pass later.	This is your heaviest day. Building edge-case handling <i>into</i> the processing layer as you write it is faster than writing the engine once and retrofitting edge cases into it on Day 5 under more time pressure.
Day 4	FastAPI backend, deploy to Render — including the API-key auth middleware and OpenAPI docs setup for the NSO/RBI-facing policy API (§8a). Start the dashboard: Next.js scaffold, Tailwind setup, wire the first 2-3 chart panels to real API data (national APIx trend, route ranking — skip the fancier ones for now).	Backend must exist before the dashboard can show real data, but the dashboard scaffold/Tailwind setup can start the moment Render deployment is confirmed working, same day.
Day 5	Globe component (<code>react-globe.gl</code>), remaining chart panels (hikes/drops, lead-time curve, DAPIx/IAPix split), Framer Motion polish. Run the DGCA backtest validation. Write <code>methodology.md</code> , <code>edge_cases.md</code> , <code>limitations.md</code> , <code>judge_qa_prep.md</code> .	Visual polish and documentation are last because they depend on the pipeline actually producing real numbers first — no point animating a chart with no data behind it yet.
Day 6 (if you have it)	Full demo rehearsal, bug fixes, re-check every judge-question answer against what the live system actually shows.	If you only have 5 days total, fold this into the second half of Day 5 — but treat rehearsal as non-negotiable even if compressed to 2-3 hours, since an unrehearsed live demo is the single most common way a strong project loses points.

Base period resolution: Day 1's 2-3 test collection runs (median per route/window) become your reference price — not a 5-7 day window. Label this explicitly in `methodology.md`:

"Prototype base period established from initial-day collection; a production system would use a longer, more statistically robust reference period." This is an honest, defensible framing, not a hidden shortcut.

1. Architecture overview — where each piece actually runs

Two changes simplify this a lot from the earlier version: collectors are hosted (Option B), and Pentaho is gone entirely — ingestion, normalization, and dedup are now plain Python, merged into the same pipeline as the collectors. That means almost everything now runs in one place: a Render Cron Job, on a schedule, independent of your laptop.

CLOUD (all of it)

Render Cron Job – runs every 4 hours, both domestic and international

1. Collectors (Python)

Bright Data adapter

Scrappa adapter

OTA Playwright scraper

2. Ingestion/Normalization/Dedup (Python, same job, runs right after collection)

airport code lookup

fare-family tier lookup + FSID construction

dedup (codeshare-aware)

3. Writes directly to Postgres (Neon, free tier)

observations, routes,
route_index, national_index



Python processing (separate Render Cron

Job,

runs right after the ingestion job

finishes)

quality/FX engine

imputation hierarchy

index engine (Jevons/Young)



FastAPI backend (Render Web Service,

always-on)



React dashboard (Vercel, free tier)

YOUR WINDOWS LAPTOP

Only used for writing/testing code before you push it – nothing has to stay running here.

Why this is simpler than before: with Pentaho gone and collectors hosted, there's no more "your laptop must be on" dependency anywhere in the pipeline. Your laptop is purely a development machine now — write code, test it, `git push`, Render picks it up. This also means you can build from anywhere, not just your desk.

Two Render Cron Jobs, not one, and here's why: the collection+ingestion job and the processing (quality/index) job run as separate scheduled jobs rather than one big script, so a failure in one doesn't silently corrupt the other — if Bright Data times out mid-run, that

failure is contained to the collection job, and the processing job simply works with whatever data made it into Postgres, applying the imputation hierarchy for whatever's missing.

1a. Frontend visual design spec — inspired by Upgraded Points' U.S. Airfare Watch Index

Pulled the reference site you linked. It's a data-journalism-style layout — national trend chart, most/least expensive airport rankings, most/least expensive routes, and a "biggest hikes and drops" percentage-change view. Here's how that maps onto your dashboard, plus the interactive globe on top of it:

Stack additions:

- **TailwindCSS** — utility-first styling, keeps the dashboard fast to iterate on given your timeline.
- **Framer Motion** — panel transitions, number count-up animations on the APIx figure, smooth hover states on the globe and charts.
- **Globe library:** `react-globe.gl` (built on Three.js, already a React component — far less setup than raw Three.js for this use case). Renders your 12+2 routes as arcs between airports on a rotatable, draggable 3D globe.

Globe interaction:

- User can freely rotate/drag the globe.
- Each route renders as an arc between its two airports, color-coded by that route's current price-relative (e.g., green = cheaper than base, red = more expensive) — same color logic as your route heatmap, just in 3D.
- Hovering or clicking an arc pops up a small card: route name, current APIx contribution, today's price relative, direct/CPI-compatible badge.
- This becomes your literal opening screen for the judge demo — "here's what's happening right now, spin the globe."

Chart panels, mapped from the reference site's structure to your actual data:

Their panel	Your equivalent	Chart type
"The Average Airfare in the U.S." (national trend)	National APIx daily/monthly trend	Line chart, LIVE/MTD/FINALIZED badge per point
"Most/Least Expensive Flight Routes"	Route ranking by current price relative	Horizontal bar chart, sorted
"Largest Airfare Hikes and Drops"	Route-level day-over-day % change	Diverging bar chart (hikes right/red, drops left/green)
Airport-level breakdown	Domestic vs. international split (DAPIx vs IAPIx)	Side-by-side small multiples
(new — not on their site)	Lead-time curve (T+1 through T+60)	Line chart, one line per route or an average curve
(new — not on their site)	Coverage/confidence score	Simple gauge or badge, not a full chart

One thing to flag: Upgraded Points' page is built on quarterly government data — their charts are static-feeling because the underlying data barely moves. Yours updates every 4-6 hours, so lean into that difference in the design — the count-up animation on the live APIx number, the globe's arc colors visibly shifting between page loads, is exactly the "real-time" story that they can't tell but you can.

2. Accounts and signups — do these first, in this order

2.1 GitHub (repo host)

1. Create a new **private** repo: `apix-sih-ps26056`.
2. Since you're solo, keep it simple: `main` branch only, commit directly. No need for a branching workflow.

2.2 Neon (cloud Postgres — free tier)

1. Go to neon.tech, sign up (GitHub login is easiest).
2. Create a new project — name it `apix-sih`.
3. Neon gives you a **connection string** immediately (looks like `postgresql://user:pass@ep-xxxx.neon.tech/apix`). Copy this — you'll need it in two places: your Python `.env` (for local testing) and Render's environment variables (for the deployed cron jobs and API).
4. Neon's free tier gives you enough storage and compute for this build's data volume comfortably.

2.3 Bright Data (primary flight data source)

1. Sign up at brightdata.com — no credit card required for the free 5,000 credits/month.
2. In the dashboard, create a new **zone** specifically for the Google Flights Scraper API product (not the generic Web Unlocker — pick the dedicated Google Flights scraper if listed as a separate product; if not listed separately, use their SERP API zone type, which covers Google Flights).
3. Generate an **API token** from Account Settings → API Keys. Copy it somewhere safe — you'll paste it into your collector's `.env`, never into code directly.

2.4 Scrappa (secondary source)

1. Sign up at scrappa.co (or the current domain — verify at signup time since these smaller providers occasionally change domains).
2. Get your free-tier API key from their dashboard.

2.5 Render (backend + collectors hosting — free tier)

1. Sign up at render.com, connect your GitHub account so it can deploy directly from your repo.
2. Don't create the actual services yet — we'll do that once the code exists. You'll eventually create three things here: one Cron Job for collection+ingestion (every 4 hours), one Cron Job for processing/index calculation (runs right after), and one Web Service for FastAPI (always-on). For now, just have the account ready.

2.6 Vercel (frontend hosting — free tier)

1. Sign up at vercel.com, connect GitHub the same way.
2. Same as Render — account ready, deploy later once the dashboard code exists.
3. When you deploy, name the Vercel project `flywise` (or `flywise-apix` if that's taken) — this becomes your demo URL, e.g. `flywise.vercel.app` or `flywise-apix.vercel.app`.

2.7 RBI/FBIL exchange rates — no signup needed

FBIL publishes daily reference rates publicly. No login required; your FX-lock script will fetch this on a schedule. (We'll confirm the exact endpoint/page format when we get to that script.)

3. Windows-specific installs

3.1 Python

Install Python 3.11+ from python.org (check "Add to PATH" during install). Verify with `python --version` in Command Prompt.

3.2 Node.js (for the React dashboard, later)

Install Node.js LTS from nodejs.org. Verify with `node --version`.

3.3 Git for Windows

Install from git-scm.com if not already present.

4. Repo folder structure

```
apix-sih-ps26056/
├── collectors/
│   ├── bright_data_adapter.py
│   ├── scrappa_adapter.py
│   ├── ota_scraper.py
│   └── render_cron_entrypoint.py    # single entrypoint the Render Cron Job calls
every 4 hrs
│   └── raw_output/                # JSON dumps land here before ingestion picks
them up
│       ├── domestic/
│       └── international/
│
├── ingestion/                    # Python – replaces Pentaho entirely
│   ├── ingest_raw_json.py
│   ├── normalize_airports.py
│   ├── normalize_fare_family.py    # builds fare_family_tier + the FSID
│   ├── dedup_observations.py      # codeshare-aware dedup
│   └── lookup_tables/             # CSV master tables the scripts read from
│       ├── airport_codes.csv
│       ├── airline_fare_family_map.csv # your CQP fare-tier mapping, per airline
│       └── route_master.csv        # the 12+2 routes, weights, seasonal flags
│
├── processing/                   # Python – quality/FX + index engine, separate
cron job
│   ├── quality_fx_engine.py        # reconciliation checks, FX-lock,
quality_score
│   ├── imputation.py              # the 4-tier imputation hierarchy
│   ├── index_engine.py            # Jevons + Young, DAPIx/IAPix split
│   └── fx_rate_fetcher.py          # pulls daily RBI/FBIL reference rates
│
├── api/                          # FastAPI, deployed to Render
│   ├── main.py
│   ├── routers/
│   │   ├── apix.py                # /apix/daily, /apix/weekly, /apix/monthly
│   │   ├── routes.py              # /routes/{route}/history
│   │   ├── coverage.py            # /coverage, /confidence
│   │   └── policy_export.py        # /policy/apix.csv, /policy/routes.csv – bulk
export for NSQ/RBI
│   ├── middleware/
│   │   └── api_key_auth.py        # checks X-API-Key header against an allowed-
keys list
│   ├── models/                   # Pydantic schemas matching the DB tables
│   └── requirements.txt
```

```

|
|— dashboard/                                # React/Next.js, deployed to Vercel
|   |— src/
|       |— components/
|           |— RouteGlobe.tsx                # react-globe.gl, arcs colored by price
relative
|       |— ApixTrendChart.tsx                # national daily/monthly line chart
|       |— RouteRankingChart.tsx             # most/least expensive routes, bar chart
|       |— HikesDropsChart.tsx              # diverging bar chart, day-over-day % change
|       |— LeadTimeCurveChart.tsx
|       |— pages/
|           |— api-client.ts
|   |— tailwind.config.js
|   |— package.json
|
|— db/
|   |— schema.sql                            # full DDL, see §5 below
|   |— seed_routes.sql                      # inserts the 12+2 routes
|   |— migrations/                          # any schema changes after initial build
|
|— docs/
|   |— methodology.md                       # judge-facing: formula, weighting, CQP
explanation
|   |— edge_cases.md                        # judge-facing: every edge case + how it's
handled
|   |— limitations.md                       # what the PS asks for that's explicitly out
of scope
|   |— judge_qa_prep.md                     # the Q&A pairs from earlier planning,
consolidated
|       |— policy_api_guide.md              # how NSO/RBI would actually consume the API —
auth, endpoints, formats
|
|— config/
|   |— .env.example                         # template — real .env is gitignored
|   |— .gitignore
|
|— README.md

```

5. Database schema — the full, consolidated version

This merges everything discussed across all our planning sessions into one schema. Run this on Neon via a plain `(psql)` connection (or Neon's own SQL editor in their web console) once the DB is created.

sql

```

-- ROUTES: the 12 core + 2 seasonal, with weights and seasonality flags
CREATE TABLE routes (
    route_id VARCHAR(10) PRIMARY KEY,          -- e.g. 'DEL-BOM'
    origin_airport VARCHAR(3) NOT NULL,
    destination_airport VARCHAR(3) NOT NULL,
    domestic_international VARCHAR(15) NOT NULL, -- 'domestic' | 'international'
    route_weight NUMERIC,                      -- DGCA-traffic-derived, prototype weight
    currency VARCHAR(3) DEFAULT 'INR',
    is_seasonal BOOLEAN DEFAULT FALSE,
    season_window VARCHAR(30)                  -- e.g. 'winter_leisure_oct_mar', NULL if not seasonal
);

-- OBSERVATIONS: one row per fare quote collected
CREATE TABLE observations (
    observation_id UUID PRIMARY KEY,
    route_id VARCHAR(10) REFERENCES routes(route_id),
    service_type VARCHAR(10) NOT NULL,          -- 'direct' | 'connect' (all core 12 are direct)
    collection_timestamp TIMESTAMP NOT NULL,
    travel_date DATE NOT NULL,
    advance_days INT NOT NULL,                 -- T+1, 7, 15, 21, 30, 45, 60
    window_category VARCHAR(20) NOT NULL,       -- 'cpi_compatible' | 'analytical'

    airline VARCHAR(10),
    flight_number VARCHAR(10),
    operating_carrier VARCHAR(10),              -- for codeshare dedup
    cabin VARCHAR(20) DEFAULT 'economy',

    fare_family_raw VARCHAR(50),
    fare_family_tier VARCHAR(20),               -- 'base' | 'standard' | 'flex' (from low to high)
    baggage_bucket VARCHAR(10),                -- '0kg' | '15kg' | '20kg+'
    service_spec_id VARCHAR(100),              -- composite FSID, see §7 for how it's built

    base_fare NUMERIC,
    taxes NUMERIC,
    mandatory_fees NUMERIC,
    total_fare_original_currency NUMERIC,
    original_currency VARCHAR(3),
    fx_rate_used NUMERIC,
    fx_rate_date DATE,
    total_fare_inr NUMERIC,                    -- computed: original * fx_rate

    availability_status VARCHAR(20),            -- 'available' | 'sold_out' | 'cancelled' | 'rebookable'
    is_imputed BOOLEAN DEFAULT FALSE,
    imputation_method VARCHAR(30),              -- NULL | 'neighbor_window' | 'time_carryover'

```

```

quality_score NUMERIC,

source VARCHAR(30), -- 'bright_data' | 'scrappa' | 'ota_playwrig
collection_method VARCHAR(30),
run_id VARCHAR(50),
raw_reference TEXT, -- pointer to stored raw JSON blob
methodology_version VARCHAR(10) DEFAULT 'v1.0'
);

-- FLIGHT EVENTS: operational disruptions, separate from pricing (edge case #1)
CREATE TABLE flight_events (
    event_id UUID PRIMARY KEY,
    flight_number VARCHAR(10),
    travel_date DATE,
    operational_status VARCHAR(20), -- 'completed' | 'diverted' | 'turned_back' | 'delayed'
    logged_at TIMESTAMP
);

-- ROUTE_INDEX: daily route-level price relative (Jevons output)
CREATE TABLE route_index (
    route_id VARCHAR(10) REFERENCES routes(route_id),
    window_category VARCHAR(20),
    advance_days INT,
    date DATE,
    price_relative NUMERIC, -- Jevons geometric mean result
    n_observations INT, -- how many comparable observations fed this
    PRIMARY KEY (route_id, window_category, advance_days, date)
);

-- NATIONAL_INDEX: the actual APIx, split by domestic/international
CREATE TABLE national_index (
    date DATE,
    window_category VARCHAR(20),
    domestic_apix NUMERIC, -- DAPIx
    international_apix NUMERIC, -- IAPIx
    overall_apix NUMERIC, -- weighted combination
    status VARCHAR(15), -- 'live' | 'mtd' | 'finalized'
    coverage_score NUMERIC,
    confidence_score NUMERIC,
    PRIMARY KEY (date, window_category)
);

```

6. Data collection layer — config, not code (per your "no code yet")

Each collector needs a config file (not hardcoded values) so you can adjust routes/frequency without touching logic later. The shape you'll want:

```
routes_config.json
{
  "domestic": [
    {"route_id": "DEL-BOM", "weight": 0.XX, "windows": [1,7,15,21,30,45]},
    ... (all 8 core domestic)
  ],
  "international": [
    {"route_id": "DEL-DXB", "currency": "AED", "windows": [1,30,60]},
    ... (all 4 core international)
  ],
  "seasonal": [
    {"route_id": "DEL-GOI", "season_window": "winter_leisure_oct_mar"},
    {"route_id": "DEL-SXR", "season_window": "summer_leisure_apr_oct"}
  ]
}
```

Frequency schedule (Render Cron Job triggers, both tranches now synced):

- Domestic routes: every 4 hours (6x/day)
- International routes: every 4 hours (6x/day) — synced with domestic, no longer once/day
- Seasonal routes: every 4 hours, only during their active season window

Budget check with this change: over a 6-day build, this is $8 \text{ domestic} \times 6 \text{ windows} \times 6\text{x/day} \times 6 \text{ days} = 1,728 \text{ requests}$, plus $4 \text{ international} \times 3 \text{ windows} \times 6\text{x/day} \times 6 \text{ days} = 432 \text{ requests}$, plus ~144 for the seasonal pair — **around 2,300 total requests across 6 days**. Comfortably inside Bright Data's 5,000/month free tier, with plenty of headroom for retries. The compressed timeline actually helps here — the original 20-day budget math no longer applies since there are far fewer total collection days now.

7. Constant-Quality Pricing — how the FSID actually gets built

Since this touches "how to structure data" directly: the `service_spec_id` field in `observations` is a computed string, built at normalization time (inside `ingestion/normalize_fare_family.py`), from:

```
FSID = route_id + "-" + cabin + "-" + service_type + "-T" + advance_days + "-" +
baggage_bucket + "-" + fare_family_tier
```

Example: `DEL-BOM-eco-direct-T21-15kg-standard`

The `(airline_fare_family_map.csv)` lookup table (the normalization script reads this) is where you hardcode each airline's actual fare names into the three tiers — you'll fill this in once you see real scraped fare names on Day 1-2, per the scope decision we locked earlier (no general classifier).

8. Ingestion pipeline — detailed step-by-step for what it actually does

This replaces the Pentaho job chain entirely — same logical steps, plain Python, deployed as part of the same Render Cron Job as the collectors.

Pipeline order (`(render_cron_entrypoint.py)` calls these in sequence):

1. `(ingest_raw_json.py)` — reads the JSON files the collectors just wrote into `(collectors/raw_output/)`. Parses raw fields into a flat structure matching the `(observations)` columns.
2. `(normalize_airports.py)` — looks up each airport code against `(lookup_tables/airport_codes.csv)` to standardize any inconsistent naming between Bright Data, Scrappa, and the OTA scraper.
3. `(normalize_fare_family.py)` — looks up each fare name against `(airline_fare_family_map.csv)`, builds `(fare_family_tier)`, and constructs the full `(service_spec_id)` string (per §7).
4. `(dedup_observations.py)` — drops duplicate rows keyed on `(route_id + travel_date + operating_carrier + collection_timestamp)` (using `(operating_carrier)`, not the marketing flight number, to catch codeshare duplicates per the edge-case design).
5. **Write step:** the pipeline's last step connects to Neon via your Python Postgres client (e.g., `(psycopg2)` or `(asyncpg)`) and inserts the cleaned rows into `(observations)`.

Deploying this as a Render Cron Job:

1. In Render's dashboard, create a new Cron Job, point it at your GitHub repo.
2. Set the schedule to every 4 hours (cron expression `(0 */4 * * *)`).
3. Set the command to run `(render_cron_entrypoint.py)`, which calls the collectors and then the ingestion scripts in order.
4. Add your Neon connection string, Bright Data API token, and Scrappa API key as environment variables in Render's dashboard — never commit these to the repo.

Second Cron Job for processing (quality/FX/index engine, §processing folder): same setup, a separate Render Cron Job, scheduled to start ~15 minutes after the ingestion job (so it always runs against that cycle's freshly-ingested data), calling `(processing/quality_fx_engine.py)` → `(processing/imputation.py)` → `(processing/index_engine.py)` in sequence.

8a. Policy/Institutional API — the PS's explicit NSO/RBI requirement

The PS's Expected Solution line says the dashboard "must... provide an API that the NSO and RBI can consume." This is a distinct requirement from your dashboard's own internal API calls — it means an external institutional consumer should be able to pull your data directly, without needing your dashboard at all. Three concrete things satisfy this cleanly:

1. API-key authentication (not open-public, not full OAuth) For a prototype, a simple shared-secret header is the right level of security — real institutional API access control, without building a full auth system you don't have time for.

- `api/middleware/api_key_auth.py` checks an `X-API-Key` header against a small allowed-keys list (stored as a Render environment variable, e.g. one demo key for judges to test with).
- Document this plainly: *"Production deployment would integrate with MoSPI/RBI's own institutional authentication; this prototype demonstrates the access-control pattern with API keys."*

2. Auto-generated OpenAPI documentation (FastAPI gives you this for free) FastAPI automatically serves interactive docs at `/docs` (Swagger UI) and `/redoc` — zero extra work, just don't disable it. This is genuinely one of the strongest "an institution could consume this" demonstrations you can show live: pull up `/docs` in front of a judge, and they can see every endpoint, its parameters, and try a real request themselves, right there.

3. Bulk export endpoints, not just the dashboard's paginated views NSO/RBI wouldn't want to click through a dashboard — they'd want to pull the whole series.

`api/routers/policy_export.py` adds:

- `GET /policy/apix.csv?window=cpi_compatible&from=...&to=...` — full APIx time series, CSV format, filterable by window/date range.
- `GET /policy/routes.csv` — full route-level breakdown for the same period.
- Both also available as `.json` for programmatic consumption (swap the extension or use an `Accept` header).

Document this as its own deliverable: `docs/policy_api_guide.md` should read like something you'd actually hand to an NSO analyst — the base URL, the API key mechanism, each endpoint with a sample request/response, and a note on rate limits (even a simple "max 100 requests/hour per key" is enough to show you've thought about institutional load, not just a demo click-through).

9. What's still genuinely undecided — flagging rather than assuming

1. **Which OTA site(s)** for the direct-scrape tranche — Ixigo vs. EaseMyTrip vs. Cleartrip. Any preference, or should I evaluate which has the lightest bot-protection right now?
-

10. Next steps

This document is the reference. When you're ready, ask me for the implementation prompt for any one piece — e.g., "give me the Antigravity prompt for the Bright Data adapter" or "give me the exact Python for `normalize_fare_family.py`" — and we build it one piece at a time from here.